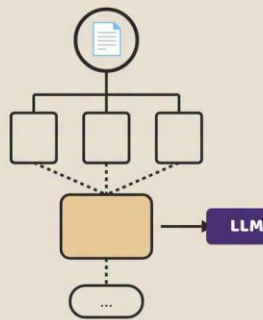


9

KHÁI NIỆM **RAG** NÂNG CAO

RETRIEVE · **AUGMENT** · GENERATE

- 🔍 Chiến lược Retrieval
- 📦 Kỹ thuật Augmentation
- ▼ Ranking & độ liên quan
- 🧠 Chất lượng Generation



1) RERANKING

Retrieval mang về các ứng viên. Reranking tìm ra cái tốt nhất.

VẤN ĐỀ

Vector DB của bạn tìm kiếm qua 100 trang tài liệu và trả về 20 chunks. Nhưng nếu câu trả lời **TỐT NHẤT** lại nằm ở vị trí #19 thì sao?

⚠️ LLM thường chỉ thấy vài chunk đứng đầu.
Câu trả lời đúng có thể không bao giờ đến được với model.

KHÔNG DÙNG RERANKING



CÓ DÙNG RERANKING



REANKER LÀM GÌ?

Nó xem xét câu truy vấn cùng với từng chunk đã retrieve, rồi trả lời:

"Chunk nào thực sự trả lời câu hỏi **tốt nhất**?"

- ✓ Hiểu ngữ cảnh tốt hơn
- ✓ Tìm ra độ liên quan ẩn
- ✓ Lọc bỏ nhiễu
- ✓ Cải thiện độ chính xác câu trả lời

CÁC LOẠI REANKER

- Cross-Encoder
(độ chính xác cao, chậm hơn)
- Bi-Encoder + Rerank Model
(cân bằng)
- LLM-based Reranker
(chất lượng cao nhất, tốn kém)

VÍ DỤ

Câu hỏi: "Làm sao để deploy FastAPI lên production?"

Không rerank:

Chunk về logging xếp #1

Có rerank:

Chunk về Unicorn + Nginx xếp #1

★ GÓC NHÌN PHÒNG VẤN

Q. Reranking trong RAG là gì?

A. Đây là kỹ thuật sắp xếp lại các chunk đã retrieve dựa trên mức độ liên quan thực sự với câu truy vấn.

💡 Ý CHÍNH

Retrieve nhiều hơn. Rerank thông minh.

Hãy để LLM thấy phần **TỐT NHẤT**, không chỉ phần đầu tiên.

2) HYBRID SEARCH

Ý nghĩa + Từ khóa. Đó là cách bạn tìm được nhiều thứ **ĐÚNG** hơn.

⚠ VẤN ĐỀ

Đôi khi **ý nghĩa** mới quan trọng.
Đôi khi **từ ngữ chính xác** mới quan trọng.
Chỉ dựa vào một loại tìm kiếm nghĩa là bạn sẽ **BỎ LỠ** kết quả tốt.

REALITY CHECK

- ✓ Người dùng không phải lúc nào cũng tìm kiếm hoàn hảo.
- ✓ Tài liệu dùng từ ngữ khác với người dùng.
- ✓ Lỗi chính tả, từ đồng nghĩa, viết tắt vẫn xảy ra.
- ✓ Bạn cần **CẢ HAI** để có kết quả tốt nhất.

Câu truy vấn: Error 500 in Nginx

Vector Search (Ngữ nghĩa)

- Khắc phục lỗi server
- Mã trạng thái HTTP
- Debug lỗi 502 bad gateway

✗ Bỏ lỡ từ khóa khớp chính xác

Keyword Search (BM25)

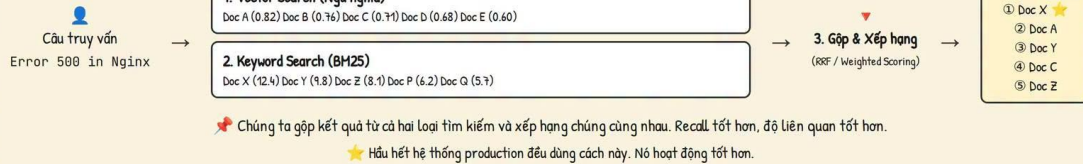
- Nginx 500 internal server error
- Sửa lỗi 500 trong Nginx
- Ví dụ Log lỗi 500

✗ Bỏ lỡ ý nghĩa / ý định

💡 HYBRID SEARCH =

Vector (hiểu ý nghĩa) + BM25 (hiểu từ khóa) = Kết quả tốt hơn

⚙️ HYBRID SEARCH HOẠT ĐỘNG NHƯ THẾ NÀO



KHI HYBRID SEARCH TỎA SÁNG

- ✓ Thuật ngữ kỹ thuật + ngôn ngữ tự nhiên
- ✓ Lỗi chính tả / viết tắt
- ✓ Tài liệu dùng từ không nhất quán
- ✓ Cần recall cao + precision cao

PHƯƠNG PHÁP KẾT HỢP PHỔ BIẾN

- Reciprocal Rank Fusion (RRF)
- Kết hợp điểm có trọng số
- Kết hợp điểm tương đối
- Kết hợp dựa trên hạng

🧐 GÓC NHÌN PHÒNG VẤN

Q. Tại sao hybrid search tốt hơn?

A. Vector search hiểu ý nghĩa nhưng có thể bỏ lỡ khớp chính xác. BM25 bắt được từ khóa chính xác nhưng bỏ lỡ ý nghĩa. Hybrid mang lại điều tốt nhất của cả hai.

💡 **Ý CHÍNH:** Đừng chọn giữa ý nghĩa hay từ khóa. Hãy dùng **CẢ HAI**. Đó là cách bạn tìm ra thứ người dùng thực sự cần.

3) CHUNKING STRATEGIES

Chunk tốt = Retrieval tốt hơn = Câu trả lời tốt hơn

🎯 TẠI SAO CHUNKING QUAN TRỌNG?

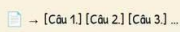
- Quá lớn → lost in the middle, kém chính xác.
- Quá nhỏ → nhiều nhiễu, ngữ cảnh kém.
- Chunk đúng → liên quan hơn, hữu ích hơn.

★ QUY TẮC VÀNG

Giữ chunk càng nhỏ càng tốt để chính xác, nhưng đủ lớn khi cần thiết để đầy đủ. 💡

CÁC CHIẾN LƯỢC CHUNKING PHỔ BIẾN

- 1 FIXED SIZE CHUNKING**
 Chia văn bản thành các chunk có độ dài cố định (vd: 500 tokens), có phần chồng lấn (overlap).

 (overlap giữa các chunk)
 - ✓ Đơn giản & nhanh
 - ✓ Hoạt động tốt nói chung
 - ✗ Có thể cắt ngang câu
- 2 SENTENCE BASED CHUNKING**
 Chia theo ranh giới câu. Mỗi chunk chứa các câu hoàn chỉnh.

 → [Câu 1] [Câu 2] [Câu 3] ...
 - ✓ Giữ nguyên ý nghĩa
 - ✓ Tự nhiên & dễ đọc
 - ✗ Độ dài câu chênh lệch nhiều
- 3 SEMANTIC CHUNKING**
 Nhóm câu/đoạn văn theo độ tương đồng ngữ nghĩa. Giữ nội dung liên quan với nhau.

 → ● Chủ đề A ● Chủ đề B ● Chủ đề C
 - ✓ Độ liên quan cao
 - ✓ Nhặt quăn chủ đề
 - ✗ Triển khai phức tạp, cần embeddings
- 4 PARENT DOCUMENT CHUNKING**
 Chia thành chunk nhỏ để retrieval, nhưng trả về phần parent lớn hơn.
 Chunk nhỏ → Parent Section (dùng cho LLM để có ngữ cảnh tốt hơn)
 - ✓ Nhiều ngữ cảnh hơn
 - ✓ Đầy đủ hơn
 - ✗ Tốn nhiều token hơn
- 5 SLIDING WINDOW CHUNKING**
 Tạo chunk bằng một cửa sổ trượt theo từng bước.
 [Window 1] [Window 2] [Window 3] ...
 - ✓ Độ phủ tốt
 - ✓ Giữ mạch ngữ cảnh
 - ✗ Nhiều chunk hơn = tốn bộ nhớ hơn
- 6 STRUCTURE AWARE CHUNKING**
 Dùng cấu trúc tài liệu (heading, list, table, code block) để chia chunk.
 # Heading 1 → Chunk 1
 # List item → Chunk 3
 # Code block → Chunk 5
 - ✓ Tôn trọng cấu trúc
 - ✓ Rất tốt cho docs/code
 - ✗ Cần parser tốt

💡 MẸO LỰA CHỌN

- ✓ Hiểu dữ liệu của bạn (docs, code, chat logs?).
- ✓ Thử các kích thước chunk khác nhau (256, 512, 1024).
- ✓ Dùng overlap (10-20%) để giữ mạch ngữ cảnh.
- ✓ Đánh giá bằng retrieval metrics, không chỉ cảm tính.

🧐 GÓC NHÌN PHONG VẤN

Q. Chiến lược chunking nào tốt nhất?
 A. Còn tùy. Không có công thức chung. Chiến lược tốt nhất là chiến lược tối đa hóa độ liên quan với nhiều tối thiểu.

🔗 THỰC TIỄN PHỔ BIẾN

Nhiều hệ thống RAG production dùng **Hybrid Approach**
 vd: Structure aware splitting + semantic grouping + parent document retrieval. ★

🔗 Chunk tốt mang lại đúng ngữ cảnh. Ngữ cảnh đúng giúp LLM đưa ra câu trả lời đúng. 🎯

4) MULTI QUERY RETRIEVAL

Một câu hỏi. Nhiều góc nhìn. Kết quả tốt hơn.

VẤN ĐỀ

Ngay cả với Query Expansion, chúng ta vẫn đang tìm kiếm theo **một hướng**. Nếu câu trả lời được viết theo cách **hoàn toàn khác** trong tài liệu thì sao?

Một query → góc nhìn hạn chế → vẫn có thể **BỎ LỖ!**

REALITY CHECK

- ✓ Các tài liệu khác nhau dùng ngôn ngữ khác nhau.
- ✓ Cùng một chủ đề được giải thích theo nhiều cách.
- ✓ Một query viết lại duy nhất vẫn có thể bỏ lỡ chunk.
- ✓ Nhiều góc nhìn hơn = recall cao hơn = câu trả lời tốt hơn.

MULTI QUERY RETRIEVAL HOẠT ĐỘNG NHƯ THẾ NÀO

1. Câu hỏi gốc

"JWT authentication hoạt động như thế nào?"

2. LLM tạo ra nhiều Query

LLM tạo ra các query đa dạng

- What is JWT?
 - JWT token được xác minh như thế nào?
 - Luồng JWT authentication
 - Access token vs Refresh token
 - JWT trong REST API security
- (Các góc nhìn khác nhau)

3. Tìm tất cả Query

- Vector Search
- Vector Search
- Vector Search
- Vector Search
- Vector Search

4. Gộp & Rerank

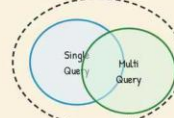
- Top Chunk cuối cùng
- ✓ Recall cao hơn
- ✓ Ngữ cảnh liên quan hơn
- ✓ Câu trả lời cuối tốt hơn

Thay vì chạy 1 query thông minh, bạn chạy **nhiều** query thông minh. Nhiều góc nhìn khác nhau → nhiều cơ hội tìm ra thông tin đúng hơn.

MULTI QUERY vs QUERY EXPANSION

Tiêu chí	Query Expansion	Multi Query
Trọng tâm	Cải thiện từ ngữ	Góc nhìn khác nhau
Query	Tương tự nhau	Đa dạng
Recall	Tốt	Cao hơn nhiều 🌟
Use case	Tổng quát	Production RAG

TRỰC QUAN



🔍 Tìm ra viên ngọc ẩn!

KHI NÓ TỎA SÁNG

- ✓ Knowledge base lớn / lộn xộn
- ✓ Tài liệu kỹ thuật dùng thuật ngữ đa dạng
- ✓ Tài liệu doanh nghiệp / đa lĩnh vực
- ✓ Bài báo nghiên cứu / pháp lý / y tế
- ✓ Bất kỳ hệ thống RAG production nào

TRIỂN KHAI PHỔ BIẾN

- LlamaIndex - MultiQueryRetriever
- LangChain - MultiQueryRetriever
- Custom LLM prompt-based retriever
- Kết hợp Reranking để có kết quả tốt nhất

GÓC NHÌN PHÒNG VẤN

Q. Tại sao dùng multi query retrieval?

A. Vì một query có thể bỏ lỡ thông tin do cách diễn đạt khác nhau. Nhiều query nhìn từ nhiều góc độ và cải thiện recall.



Ý CHÍNH: Đừng chỉ hỏi một lần. Hãy hỏi theo nhiều cách thông minh. Nhiều góc nhìn hơn → Nhiều tri thức hơn.

5) PARENT DOCUMENT RETRIEVAL

Chunk nhỏ = tìm kiếm tốt hơn. Parent doc = hiểu tốt hơn.

VẤN ĐỀ

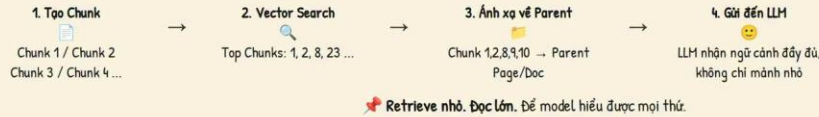
Bạn retrieve một chunk nhỏ trông có vẻ liên quan nhưng lại **thiếu ngữ cảnh đầy đủ**.
Bản thân chunk đó không có ý nghĩa gì cả.

Nhà câu chuyện ≠ Sự thật.

REALITY CHECK

- ✓ Chunk bị mất ngữ cảnh.
- ✓ Đại từ, tham chiếu bị mất.
- ✓ Số liệu, thuật ngữ trông ngẫu nhiên.
- ✓ LLM dễ hiểu sai.
- ✓ Nhiều ngữ cảnh hơn = câu trả lời rõ ràng, chính xác hơn.

PARENT DOCUMENT RETRIEVAL HOẠT ĐỘNG NHƯ THẾ NÀO



VÍ DỤ

Câu hỏi: "Tại sao FastAPI lại trở nên phổ biến?"

Chunk retrieve (thiếu ngữ cảnh):

"...nó nhanh, dễ dùng, và có docs tự động..." ✗ Không có ý nghĩa khi dùng một mình.

Parent Document (ngữ cảnh đầy đủ):

"...FastAPI trở nên phổ biến vì nó nhanh, dễ dùng, có docs tự động, hỗ trợ async, và trải nghiệm lập trình viên tuyệt vời..." ✓ Giờ thì có ý nghĩa rồi!

KHI NÀO NÊN DỪNG

- ✓ Khi chunk rất nhỏ (vd: 100-300 tokens)
- ✓ Khi tài liệu có nhiều tham chiếu
- ✓ Khi câu trả lời cần ngữ cảnh đầy đủ (không chỉ một dòng)

Chunk nhỏ
để TÌM KIẾM

Parent doc
để CÒ NGỮ CẢNH

MẸO

- ✓ Lưu parent_id cùng mỗi chunk.
- ✓ Dùng cấp page/section làm parent.
- ✓ Tránh gửi quá nhiều parent.
- ✓ Cân bằng kích thước ngữ cảnh và độ liên quan.

PHỔ BIẾN TRONG

- Hệ thống RAG
- Chatbot cho PDF dài
- Trợ lý pháp lý / nghiên cứu
- Knowledge base

GÓC NHÌN PHÒNG VẤN

Q. Tại sao không gửi thẳng chunk?

A. Chunk bị mất ngữ cảnh. Parent doc mang lại những phần còn thiếu giúp câu trả lời chính xác.

💡 **KẾT LUẬN:** Chunk giúp bạn **TÌM**. Parent doc giúp model **HIỂU**. Cả hai kết hợp cho **câu trả lời tốt nhất**. 🎯

6) CONTEXT COMPRESSION

Quá nhiều ngữ cảnh = **lost in the middle**. Compress lại. Giữ những gì **quan trọng**.

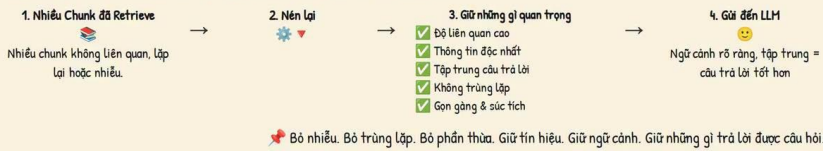
VẤN ĐỀ

Retriever của bạn tìm được 40 chunks. Nhưng LLM chỉ nhận được 8. Nếu gửi hết, model sẽ bị phân tán, thông tin quan trọng bị chôn vùi, và chất lượng câu trả lời **giảm sút**.

REALITY CHECK

- ✓ LLM có context window giới hạn.
- ✓ Thông tin dư thừa lãng phí token.
- ✓ Phần giữa bị bỏ qua (Lost in the middle).
- ✓ Chi tiết không liên quan làm model bối rối.
- ✓ Compress giúp cải thiện độ tập trung, độ chính xác & chi phí.

CONTEXT COMPRESSION HOẠT ĐỘNG NHƯ THẾ NÀO



CÁI GÌ BỊ NÉN?

- 📄 Chunk trùng lặp
- 💬 Câu thừa
- ✗ Độ liên quan thấp
- ✂ Chi tiết dài dòng
- 📄 Phần lạc đề

CÁCH NÉN PHỔ BIẾN

- 🌟 LLM Summarization
Tóm tắt mỗi chunk trong 1-2 dòng.
- 🔑 Keyword/Keyphrase
Chỉ trích xuất cụm từ quan trọng.
- 🗑 Redundancy Removal
Loại bỏ chunk giống/lặp lại.
- ✂ Extractive Compression
Giữ câu quan trọng, bỏ phần còn lại.
- 📊 Relevance Scoring
Chỉ giữ câu/chunk điểm cao.

VÍ DỤ

Retrieve 40 chunks



Sau nén: 8 chunks



Token usage ~75%



Chất lượng câu trả lời +

TÌNH HUỐNG THỰC TẾ

Bạn hỏi: "TastAPI xử lý background task thế nào?"
Retriever trả về 40 chunks (docs, code, ví dụ...)
Compressor loại bỏ ví dụ, lặp lại & phần lạc đề
LLM nhận 8 chunk gọn gàng, liên quan và trả lời **hoàn hảo**. 🌟

GÓC NHÌN PHÒNG VẤN

Q. Tại sao phải nén trước khi gửi cho LLM?

A. Để giảm nhiễu, tiết kiệm token, tránh lost-in-the-middle, và cải thiện chất lượng câu trả lời.

MẸO

- ✓ Tinh chỉnh mức nén cho use case của bạn.
- ✓ Đo bằng chất lượng câu trả lời, không chỉ token.
- ✓ Compression > model to hơn (đa số trường hợp).
- ✓ Ngữ cảnh nhỏ, sạch tốt hơn ngữ cảnh to, nhiễu.

Ý CHÍNH

Nhiều ngữ cảnh hơn **không phải** lúc nào cũng tốt hơn.
Ngữ cảnh **liên quan** mới tốt.
Nén thông minh. Trả lời tự tin.

💡 Câu trả lời tốt nhất đến từ **ngữ cảnh đúng**, không phải **ngữ cảnh lớn**. 🌟

7) HYDE (HYPOTHETICAL DOCUMENT EMBEDDINGS)

Đôi khi câu hỏi **quá mơ hồ** để tìm kiếm tốt.
Vì vậy... ta **tưởng tượng** một câu trả lời hoàn hảo, rồi **tìm kiếm** dựa trên đó.

VẤN ĐỀ

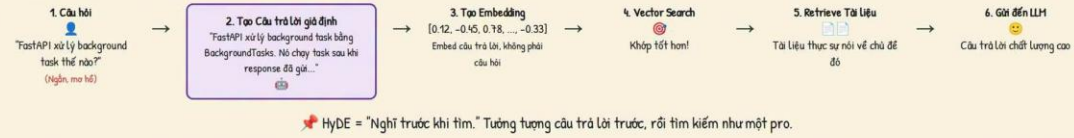
Câu hỏi của bạn quá ngắn hoặc mơ hồ. Tìm kiếm trực tiếp có thể bỏ lỡ tài liệu tốt nhất vì query không chứa đúng từ ngữ hoặc ngữ cảnh.

Query mơ hồ → Tìm kiếm yếu → Kết quả kém

REALITY CHECK

- ✓ Query mơ hồ có embedding yếu.
- ✓ Thiếu các thuật ngữ quan trọng.
- ✓ Retrieval bỏ quên điều quan trọng.
- ✓ Biểu diễn query tốt hơn = kết quả tốt hơn.
- ✓ HyDE tạo ra biểu diễn đó cho bạn.

HYDE HOẠT ĐỘNG NHƯ THẾ NÀO



KHÔNG PHẢI ĐOÁN BÙA!

HyDE không bịa ra ngẫu nhiên.

Nó tạo ra câu trả lời giả định có cấu trúc, liên quan, am hiểu domain.

Nó nắm bắt ý nghĩa bạn định nói nhưng chưa diễn đạt được. 🤖

VÍ DỤ

Query: "FastAPI background tasks"

Tìm trực tiếp (kém)

top kết quả về Celery, threading, cron jobs... ❌

Tìm bằng HyDE (tốt hơn)

top kết quả về BackgroundTasks, add_task, execution... ✓

TAI SAO HIỆU QUẢ

- ✓ Dùng thuật ngữ có ý nghĩa hơn.
- ✓ Nắm bắt ngữ cảnh và ý định.
- ✓ Cải thiện recall đáng kể.
- ✓ Tốt cho câu hỏi mơ hồ, phức tạp.
- ✓ Hữu ích cho nghiên cứu, code, pháp lý, y tế.

PHỔ BIẾN TRONG

- Hệ thống RAG
- Trợ lý nghiên cứu
- Tìm kiếm code
- Tìm kiếm pháp lý / y tế
- Knowledge base doanh nghiệp

GÓC NHÌN PHÒNG VẤN

Q. HyDE khác Query Expansion thế nào?

A. Query Expansion tạo nhiều query. HyDE tạo một câu trả lời giả định duy nhất, rồi embed nó.

MẸO

- ✓ Dùng LLM tốt để có câu trả lời giả định tốt hơn.
- ✓ Giữ phần generate ngắn gọn nhưng giàu thông tin.
- ✓ Tốt hơn khi kết hợp Reranking.
- ✓ Kết hợp Hybrid Search để recall tối đa.

💡 HyDE biến một câu hỏi **yếu** thành tín hiệu tìm kiếm **mạnh**. Tín hiệu tốt hơn → Retrieval tốt hơn → Câu trả lời tốt hơn. ⭐

8) SELF-RAG (SELF REFLECTIVE RAG)

Tại sao lúc nào cũng phải tìm kiếm? Hãy để **model tự quyết định** trước.

VẤN ĐỀ

Trong RAG thông thường, ta **luôn luôn retrieve**, kể cả với câu hỏi đơn giản. Điều này lãng phí **thời gian, token và tiền bạc**. Nếu model **đã biết sẵn** thì sao?

REALITY CHECK

- ✓ Nhiều câu hỏi không cần tìm kiếm bên ngoài.
- ✓ Retrieval không cần thiết = tốn thêm chi phí + độ trễ.
- ✓ Self-RAG để model suy nghĩ trước khi hành động.
- ✓ Hiệu quả hơn. Câu trả lời tốt hơn.
- ✓ Đúng công cụ, đúng thời điểm!

SELF-RAG HOẠT ĐỘNG NHƯ THẾ NÀO



VÍ DỤ

Q1: 2 + 2 bằng mấy?
→ Model: "Mình biết câu này."
→ Trả lời trực tiếp ✓

Q2: Release note mới nhất của FastAPI?
→ Model: "Mình không chắc."
→ Retrieve → Rồi trả lời ✓ 📄

LỢI ÍCH

- ✓ Giảm retrieval không cần thiết
- ✓ Tiết kiệm token & chi phí
- ✓ Phản hồi nhanh hơn
- ✓ Giảm tải cho Vector DB
- ✓ Câu trả lời tốt hơn (khi cần) 🚀

KHI QUYẾT ĐỊNH "KHÔNG"

- Câu hỏi kiến thức chung
- Toán / Logic / Sự kiện đơn giản
- Câu hỏi thông thường (common sense)
- Câu hỏi model tự tin

KHI QUYẾT ĐỊNH "CÓ"

- Thông tin gần đây / thời gian thực
- Kiến thức chuyên biệt theo domain
- Tài liệu riêng tư / nội bộ
- Câu hỏi phức tạp, multi-hop

PHỔ BIẾN TRONG

- Chatbot
- Trợ lý AI
- Tìm kiếm doanh nghiệp
- Helpdesk, Knowledge base

GÓC NHÌN PHÒNG VẤN

Q. Self-RAG khác RAG truyền thống thế nào?
A. RAG truyền thống **luôn retrieve**. Self-RAG **quyết định trước** có cần retrieval không, rồi mới hành động. → Thông minh hơn. Nhanh hơn. Rẻ hơn.

💡 Self-RAG không chỉ retrieve... nó **suy nghĩ trước**. Quyết định thông minh → Hiệu quả tốt hơn → Câu trả lời tốt hơn. ★

9) CRAG (CORRECTIVE RAG)

Không phải chunk nào retrieve được cũng hữu ích. **CRAG** kiểm tra **chất lượng** trước khi tin dùng.

VẤN ĐỀ

Retriever không hoàn hảo. Nó có thể trả về chunk không liên quan, lỗi thời, hoặc gây hiểu lầm. Nếu LLM tin vào ngữ cảnh xấu, nó sẽ đưa ra **câu trả lời sai với sự tự tin tuyệt đối**.

REALITY CHECK

- ✓ Retriever có thể bỏ lỡ thông tin liên quan nhất.
- ✓ Kết quả top có thể trông liên quan nhưng lại sai.
- ✓ LLM khó tự đánh giá chất lượng retrieval.
- ✓ CRAG thêm bước kiểm tra chất lượng + vòng lặp sửa lỗi.
- ✓ Kiểm định tốt hơn = câu trả lời tốt hơn.

CRAG HOẠT ĐỘNG NHƯ THẾ NÀO



VÍ DỤ

Query: "Database nào tốt nhất cho real-time analytics?"

Retriever trả về:

1. SQLite tutorial (nhiều)
2. MongoDB basics (lỗi thời)
3. Clickhouse real-time case study ✓
4. Postgres setup (liên quan một phần)

CRAG kiểm tra → loại doc yếu → tìm doc tốt hơn → trả lời chính xác. 

LỢI ÍCH

- ✓ Ngắn câu trả lời sai nhưng tự tin.
- ✓ Cải thiện độ chính xác câu trả lời.
- ✓ Giảm hallucination.
- ✓ Xử lý query phức tạp tốt hơn.
- ✓ Tiết kiệm token nhờ lọc nhiễu. 

KHI CRAG HỮU ÍCH NHẤT

- Câu hỏi phức tạp / multi-hop
- Query có nhiều cách hiểu
- Khi chất lượng retriever thấp
- Tìm kiếm kỹ thuật, chuyên ngành
- Query mơ hồ, hiểm hóc

PHỔ BIẾN TRONG

- Hệ thống RAG nâng cao
- Tìm kiếm doanh nghiệp
- Trợ lý nghiên cứu
- AI pháp lý / y tế / tài chính
- Ứng dụng AI production-grade

GÓC NHÌN PHÒNG VẤN

Q. CRAG khác RAG thông thường thế nào?

A. RAG thường: Retrieve → Answer (không kiểm tra chất lượng).

CRAG: Retrieve → Evaluate → Correct → Answer.

CRAG thêm vòng lặp kiểm định + sửa lỗi để đảm bảo chất lượng trước khi trả lời.

MẸO

- ✓ Dùng evaluator mạnh (LLM/cross-encoder).
- ✓ Định nghĩa tiêu chí chất lượng rõ ràng.
- ✓ Đặt ngưỡng để loại bỏ.
- ✓ Tinh chỉnh chiến lược re-query hợp lý.
- ✓ Kết hợp Reranking & Hybrid Search.

 CRAG không chỉ tin retriever... nó **xác minh, sửa lỗi** và đảm bảo model thấy ngữ cảnh tốt nhất. Kiểm định tốt hơn → Retrieval tốt hơn → Câu trả lời tốt hơn. 